



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - СОФИЯ
ФИЛИАЛ ПЛОВДИВ

ФАКУЛТЕТ ПО ЕЛЕКТРОНИКА И АВТОМАТИКА
КАТЕДРА “КОМПЮТЪРНИ СИСТЕМИ”

ПРОЕКТИРАНЕ НА СВРЪХГОЛЕМИ ИНТЕГРАЛНИ
СХЕМИ

Протокол № 4

Изготвил: Любомир Ламбрев

Група: 141

Факултетен номер: 614973

Datapath

1. Теоретична част

Datapath е основен компонент в цифровите системи, който реализира предаването и обработката на данни. Той съдържа функционални блокове като регистри, мултиплексори, аритметично-логически устройства (ALU), и шини, които заедно позволяват изпълнението на различни операции.

Datapath работи чрез взаимодействие между входни сигнали и вътрешни модули:

- Данните се подават на входа (DataInput).
- Сигналят IE управлява дали входните данни ще бъдат използвани от ALU.
- ALU извършва операция, избрана чрез ALU_Sel (3-битов вход).
- Резултатът от ALU може да бъде записан в регистър при активен сигнал Load.
- Чрез OE, съдържанието на регистъра се извежда на изхода DataOut.
- Сигналят Clear служи за нулиране на регистъра.

Описание на използваните модули

- **ALU (аритметично-логическо устройство):** Извършва операции като AND, OR, NOT, ADD, SUB и др.
- **Register8:** 8-битов регистър с входове за запис (Load), нулиране (Clear) и изходно разрешение (OE).
- **MUX_Input:** Мултиплексор, избиращ между DataInput и ALU резултата.
- **Output_Enable:** Управлява дали Reg_Out се подава към DataOut.

Очаквани резултати

стъпка	Clock	Clear	Load	OE	IE	ALU_Sel	DataInput	Резултат
1	1	1	0	0	0	000	00000000	Нулиране
2	1	0	1	0	1	000	00001010	Reg_out = 10
3	1	0	1	0	1	100	00000101	ALU = 5 + 10
4	1	0	0	1	0	000	00000000	Dataout = 15

2. Описание на VHDL код

Datapath.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Datapath is
  Port (
    Clock, Clear, Load, OE, IE : in  STD_LOGIC;
    ALU_Sel : in  STD_LOGIC_VECTOR(2 downto 0);
    DataInput : in  STD_LOGIC_VECTOR(7 downto 0);
    DataOut : out STD_LOGIC_VECTOR(7 downto 0)
  );
end Datapath;

architecture Structural of Datapath is
  signal A, B, ALU_Result, Reg_Out : STD_LOGIC_VECTOR(7 downto 0);
begin
  -- Входен мултиплексор
  MUX1 : entity work.MUX_Input port map (
    IE => IE,
    Data => DataInput,
    OutA => A
  );

  -- ALU
  U_ALU : entity work.ALU port map (
    A => A,
    B => Reg_Out,
    ALU_Sel => ALU_Sel,
    ALU_Out => ALU_Result
  );

  -- Регистър
  U_Reg : entity work.Register8 port map (
    D => ALU_Result,
    Clock => Clock,
    Clear => Clear,
    Load => Load,
```

```

        Q    => Reg_Out
    );

    -- OE контрол
    U_OE : entity work.Output_Enable port map (
        OE    => OE,
        Q     => Reg_Out,
        OutData => DataOut
    );
end Structural;

```

Output_enable.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Output_Enable is
    Port (
        OE    : in  STD_LOGIC;
        Q     : in  STD_LOGIC_VECTOR(7 downto 0);
        OutData : out STD_LOGIC_VECTOR(7 downto 0)
    );
end Output_Enable;

architecture Behavioral of Output_Enable is
begin
    OutData <= Q when OE = '1' else (others => 'Z');
end Behavioral;

```

MUX_Input.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX_Input is
    Port (
        IE  : in  STD_LOGIC;
        Data : in  STD_LOGIC_VECTOR(7 downto 0);

```

```

        OutA : out STD_LOGIC_VECTOR(7 downto 0)
    );
end MUX_Input;

```

```

architecture Behavioral of MUX_Input is
begin
    OutA <= Data when IE = '1' else "00000001";
end Behavioral;

```

Register8.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX_Input is
    Port (
        IE   : in  STD_LOGIC;
        Data : in  STD_LOGIC_VECTOR(7 downto 0);
        OutA : out STD_LOGIC_VECTOR(7 downto 0)
    );
end MUX_Input;

```

```

architecture Behavioral of MUX_Input is
begin
    OutA <= Data when IE = '1' else "00000001";
end Behavioral;

```

ALU.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity ALU is
    Port (
        A, B   : in  STD_LOGIC_VECTOR(7 downto 0);
        ALU_Sel : in  STD_LOGIC_VECTOR(2 downto 0);
        ALU_Out : out STD_LOGIC_VECTOR(7 downto 0)
    );
end ALU;

```

architecture Behavioral of ALU is

begin

process(A, B, ALU_Sel)

begin

case ALU_Sel is

when "000" => ALU_Out <= A; -- Pass through A

when "001" => ALU_Out <= A and B; -- A AND B

when "010" => ALU_Out <= A or B; -- A OR B

when "011" => ALU_Out <= not A; -- NOT A

when "100" => ALU_Out <= A + B; -- A + B

when "101" => ALU_Out <= A - B; -- A - B

when "110" => ALU_Out <= A + "00000001"; -- A + 1

when "111" => ALU_Out <= A - "00000001"; -- A - 1

when others => ALU_Out <= (others => '0');

end case;

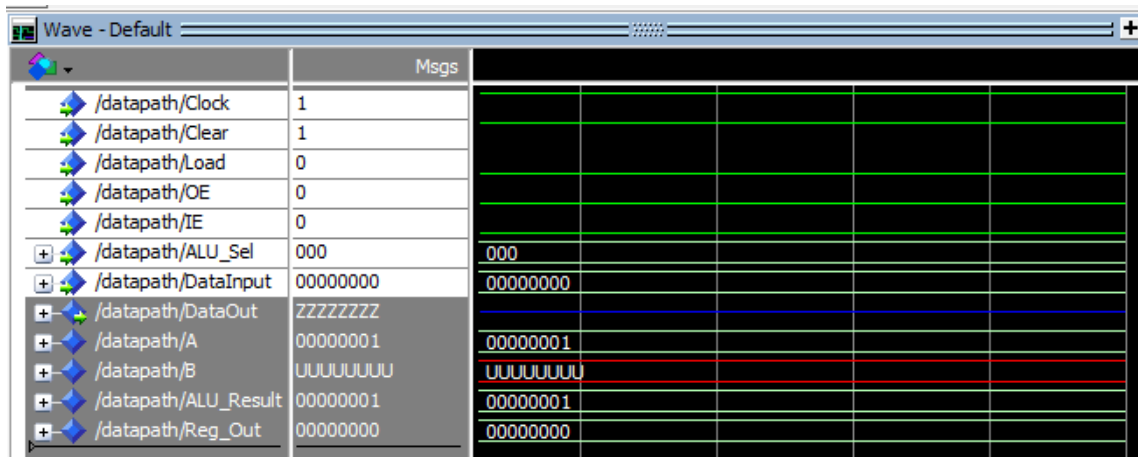
end process;

end Behavioral;

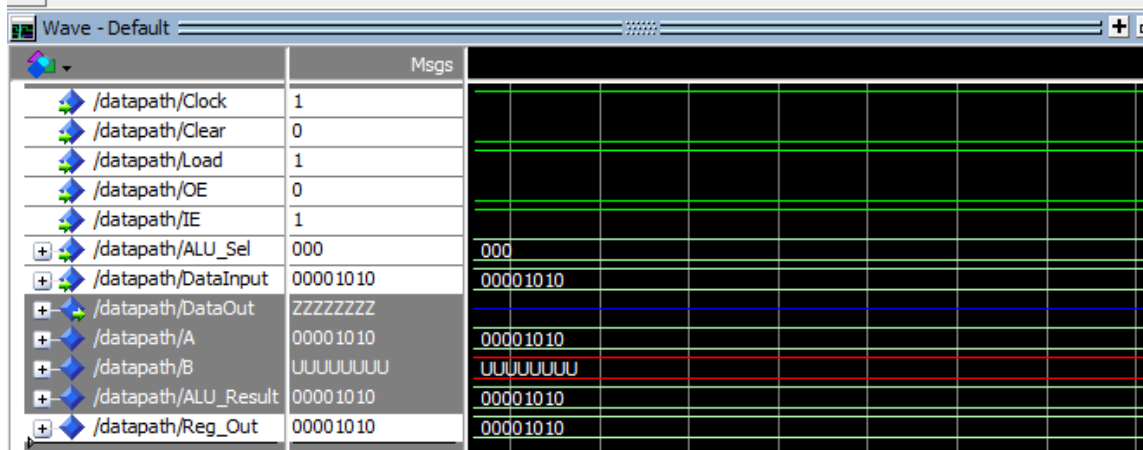
3. Симулиране и времедиаграми

Симулацията генерира следните резултати:

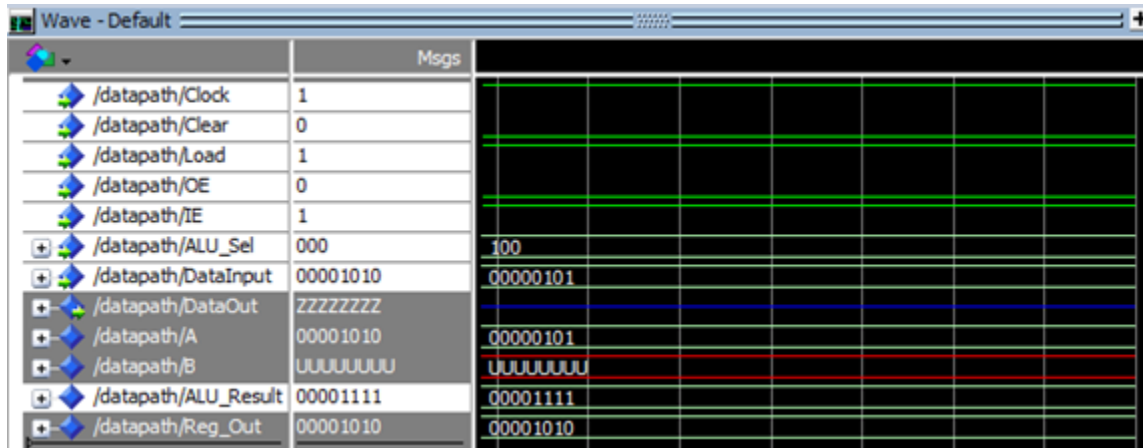
1-ва стъпка:



2-ра стъпка:



3-та стъпка:



4-та стъпка:

